

Analysis: Bathroom Stalls

Test set 1

For test set 1, the limits are small enough that you can just simulate the rules outlined in the statement. Most implementations of a simulation will run in $O(NK)$ time and thus finish immediately, but even a slow $O(N^2K)$ implementation like "try every possible stall for the next person, and for each empty stall run a loop for each side to check for the closest neighbors" will most likely finish in time.

For test sets 2 and 3, however, something quadratic in the number of stalls won't cut it, so we have to do better.

Test set 2

The critical observation to jump from test set 1 to test set 2 is that only the number of consecutive runs of empty stalls matters at any given time. The next person always chooses the middle stall or the left of the two middle stalls of a longest subsequence of consecutive empty stalls. Moreover, the output format already hints at this: even if you were to choose the rightmost of a set of two middle stalls, or a longest run of stalls other than the leftmost one, the answer would not change. Thus, we can rewrite the algorithm in this equivalent (for the required output) form:

1. Find any longest subsequence of consecutive empty stalls.
2. Choose the middle or one of the two middle stalls.

Notice that even though there are still ties to be broken, the output is equivalent for all of them. Since the output is equivalent, so is the multiset of lengths of consecutive runs of empty stalls left behind, so the whole process only depends on that multiset. (As a reminder, a multiset is a set in which the same element can appear more than once.) We can write an optimized simulation that solves test set 2 following this pseudocode:

```
S = {N} - This is a multiset!
repeat K times:
    X = max(S)
    X0 = ceil((X - 1) / 2)
    X1 = floor((X - 1) / 2)
    if this is the last step:
        we are done; answer is X0 and X1
    else:
        remove one instance of X from S
        insert X0 and X1 into S
```

If the operations over S are efficient, this will run in quasilinear time. There are many data structures that support insertion, finding the maximum, and removal of the maximum in logarithmic time, including AVL trees, red-black trees, and heaps. Many languages have one such structure in their standard libraries (e.g., the `multiset` or `priority_queue` in C++, `TreeSet` in Java, and `heapq` module in Python). Since we take $O(\log K)$ time for each of K steps, the algorithm takes only $O(K \log K)$ time, which is fast enough to solve test set 2. However, for test set 3, even quasilinear time on K is not enough.

Test set 3

The observation required to solve test set 3 is that we are simulating similar steps over and over again. The first time a bathroom user arrives, we partition N into $\text{ceil}((N - 1) / 2)$ and $\text{floor}((N - 1) / 2)$, which means that numbers between $\text{ceil}((N - 1) / 2)$ and N will never appear in S . This hints at a logarithmic number of simulation steps.

Let's divide the work in stages. The first stage processes only N . Then, stage $i+1$ processes all of the values spawned by stage i . So, stage 2 processes up to 2 values: $\text{ceil}((i - 1) / 2)$ and $\text{floor}((i - 1) / 2)$. What about the other stages? It is not hard to prove by induction that they also process at most two consecutive values: since stage i processes two consecutive values, they are either $2x$ and $2x+1$ or $2x$ and $2x-1$, for some x (that is, one even and one odd number). Thus, the spawned values for stage $i+1$ can only be x and/or $x-1$. Since the largest value in each stage is at most half the largest value of the previous stage, there are a logarithmic number of stages. This all means that there are at most $O(\log N)$ different values that go into S at any point. Of course, some of them appear in S many, many times. So, the optimization to get the running time low enough for test set 3 is to process all repetitions of a given value at the same time, since all of them yield the same X_0 and X_1 values. We can do that by using a regular set with a separate count for the number of repetitions.

```
S = {N} - This is a set, not a multiset!
C(N) = 1
P = 0
repeat:
    X = max(S)
    X0 = ceil((X - 1) / 2)
    X1 = floor((X - 1) / 2)
    P = P + C(X)
    if P ≥ K:
        we are done; the answer is X0 and X1.
    else:
        remove X from S
        insert X0 and X1 into S
        add C(X) to the counts of X0 and X1 in C
```

Once again, we have structures that implement all the required operations in logarithmic time, yielding an $O(\log^2 N)$ running time overall. In general, adding any good dictionary implementation to the structure of choice from the test set 2 solution would work, either by plugging the dictionary functionality into the structure (like `map` in C++ or `TreeMap` in Java) or having a separate hash-table for the dictionary (which is the easiest implementation in Python).

Moreover, since we proved the population of S is at most 4 at any given time (only values from two consecutive stages can coexist in S), any implementation of set and dictionary will provide all operations in constant time, because the size of the whole structure is bounded by a constant! This makes the overall time complexity just $O(\log N)$.

This was a nice problem to put experimentation to work if your intuition was not enough. After solving test set 1, if you print the succession of values for a fixed N , you may spot the pattern of few values occurring in the set S , and from there, you can find the mathematical arguments to support the needed generalization. In harder problems in later rounds, this can become an even more important asset to tackle problems. As you can see in many parts of last year's finals [live stream](#), finalists use experimentation a lot to inspire themselves and/or validate their ideas before committing to them.

Analysis: Number Guessing

Test set 1

Since **A** = 0 and **B** = 30 in this test set, and since we get **N** = 30 tries per test case, we can simply guess every number from 1 to 30 until the judge sends back `CORRECT`.

Test set 2

In test set 2, since the answer could be anywhere in the range $(0, 10^9]$ and we still have only 30 guesses, we will use binary search.

Initially, we know the answer P is in $[1, 10^9]$, which is a big range! To cut that range by half, our first guess will be $(1 + 10^9) / 2 = 5 \times 10^8$. If the judge sends back `TOO_SMALL`, we will know that P is in $[1, 5 \times 10^8]$. Similarly, if the judge sends back `TOO_BIG`, P is in $(5 \times 10^8, 10^9]$. Otherwise, P is 5×10^8 and we are done.

We will cut that range further by making our next guess the middle number in that range. Again, based on the judge response that we get, we will know that either we have guessed P correctly, or P is in the upper or lower half of the range. We will do this repeatedly, until `CORRECT` is received.

Each time we make a wrong guess, the range that we must examine next will always be at most half the size of our previous range. So, it will take at most $\log_2 10^9 = 29.897353 < 30$ tries to guess P correctly.

Sample Solutions

This problem was intended as an opportunity to get used to our interactive judges. Here are some example solutions in all languages that we support so far:

Bash:

```
read t
for p in $(seq 1 $t); do
  read -a line
  a=${line[0]}
  b=${line[1]}
  read n
  head=$(( a+1 ))
  tail=$b
  while true; do
    mid=$(( (head+tail)/2 ))
    echo $mid
    read s
    if [[ "$s" == "CORRECT" ]]; then
      break
    elif [[ "$s" == "TOO_BIG" ]]; then
      tail=$(( mid - 1 ))
    elif [[ "$s" == "TOO_SMALL" ]]; then
```

```

        head=$(( mid + 1 ))
    else
        # Wrong answer; exit to receive Wrong Answer judgment
        exit 0
    fi
done
done

```

C:

```

#include <stdio.h>
#include <string.h>

int main() {
    int T; scanf("%d", &T);

    for (int id = 1; id <= T; ++id) {
        int A, B, N, done = 0;
        scanf("%d %d %d", &A, &B, &N);
        for (++A; !done;) {
            int mid = A + B >> 1;
            char result[32];
            printf("%d\n", mid);
            fflush(stdout);
            scanf("%s", result);
            if (!strcmp(result, "CORRECT")) done = 1;
            else if (!strcmp(result, "TOO_SMALL")) A = mid + 1;
            else B = mid - 1;
        }
    }
    return 0;
}

```

C#:

```

using System;

public class Solution
{
    static public void Main ()
    {
        int num_test_cases = Convert.ToInt32(Console.ReadLine());
        for (int i = 0; i < num_test_cases; ++i) {
            string[] lo_hi_s = Console.ReadLine().Split(' ');
            int[] lo_hi = Array.ConvertAll(lo_hi_s, int.Parse);
            int num_tries = Convert.ToInt32(Console.ReadLine());
            int head = lo_hi[0] + 1, tail = lo_hi[1];
            while (true) {
                int m = (head + tail) / 2;
                Console.WriteLine (m);
                string s = Console.ReadLine();
                if (s == "CORRECT") break;
                if (s == "TOO_SMALL")
                {

```

```

        head = m + 1;
    }
    else
    {
        tail = m - 1;
    }
}
}
}
}

```

C++:

```

#include <iostream>
#include <string>

int main() {
    int num_test_cases;
    std::cin >> num_test_cases;
    for (int i = 0; i < num_test_cases; ++i) {
        int lo, hi;
        std::cin >> lo >> hi;
        int num_tries;
        std::cin >> num_tries;
        int head = lo + 1, tail = hi;
        while (true) {
            int m = (head + tail) / 2;
            std::cout << m << std::endl;
            std::string s;
            std::cin >> s;
            if (s == "CORRECT") break;
            if (s == "TOO_SMALL")
                head = m + 1;
            else
                tail = m - 1;
        }
    }
    return 0;
}

```

Go:

```

package main

import (
    "fmt"
    "strings"
)

func main() {
    var t int
    fmt.Scanf("%d", &t)
    for i := 1; i <= t; i++ {
        var a, b, n int
    }
}

```

```

fmt.Scanf("%d %d", &a, &b)
a = a + 1
fmt.Scanf("%d", &n)
for {
    m := (a + b) / 2
    fmt.Println(m)
    var str string
    fmt.Scanf("%s", &str)
    if strings.EqualFold(str, "CORRECT") {
        break
    } else if strings.EqualFold(str, "TOO_SMALL") {
        a = m + 1
    } else if strings.EqualFold(str, "TOO_BIG") {
        b = m - 1
    }
}
}
}

```

Haskell:

```

import System.IO

getNum :: IO Int
getNum = do
    x <- getLine
    let n = read x :: Int
    return n

bisect :: Int -> Int -> Int -> String -> IO ()
bisect a b m "CORRECT" = return ()
bisect a b m "TOO_SMALL" = singleCase (m+1) b
bisect a b m "TOO_BIG" = singleCase a (m-1)

query :: Int -> IO String
query m = do
    putStrLn ( show m )
    hFlush stdout
    x <- getLine
    return x

singleCase :: Int -> Int -> IO ()
singleCase a b = do
    let m = (a+b)  $\text{\textbf{div}}$  2
    response <- query m
    bisect a b m response
    return ()

solve :: Int -> IO ()
solve 0 = return ()
solve n = do
    [a, b] <- fmap(map read.words)getLine
    _ <- getNum
    singleCase (a+1) b
    solve (n-1)

```

```

main = do
    hSetBuffering stdout NoBuffering
    t <- getNum
    solve t

```

Java:

```

import java.util.Scanner;

public class Solution {
    public static void solve(Scanner input, int a, int b) {
        int m = (a + b) / 2;
        System.out.println(m);
        String s = input.next();
        if (s.equals("CORRECT")) {
            return;
        } else if (s.equals("TOO_SMALL")) {
            solve(input, m + 1, b);
        } else {
            solve(input, a, m - 1);
        }
    }

    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        int T = input.nextInt();
        for (int ks = 1; ks <= T; ks++) {
            int a = input.nextInt();
            int b = input.nextInt();
            int n = input.nextInt();
            solve(input, a + 1, b);
        }
    }
}

```

JavaScript:

```

var readline = require('readline');
var rl = readline.createInterface(process.stdin, process.stdout);

expect = 'begin';
rl.on('line', function(line) {
    if (expect === 'begin') {
        num_test_cases = parseInt(line);
        expect = 'lo_hi';
        case_counter = 0;
    } else if (expect === 'lo_hi') {
        lo_hi = line.split(' ');
        head = parseInt(lo_hi[0]) + 1;
        tail = parseInt(lo_hi[1]);
        expect = 'num_tries';
    } else if (expect === 'num_tries') {
        num_tries = line; // not used.
    }
});

```

```

    expect = 'solve';
    mid = parseInt((head + tail) / 2);
    console.log(mid);
} else if (expect === 'solve') {
    if (line === 'CORRECT') {
        ++case_counter === num_test_cases ? rl.close() : 0;
        expect = 'lo_hi';
    } else {
        line === 'TOO_SMALL' ? head = mid + 1 : tail = mid - 1;
        mid = parseInt((head + tail) / 2);
        console.log(mid);
    }
}
}).on('close',function(){
    process.exit(0);
});

```

PHP:

```

<?php

function solve($a, $b) {
    $m = ($a + $b) / 2;
    printf("%d\n", $m);
    fscanf(STDIN, "%s", $s);
    if (strcmp($s, "CORRECT") == 0) {
        return;
    } else if (strcmp($s, "TOO_SMALL") == 0) {
        $a = $m + 1;
    } else {
        $b = $m - 1;
    }
    solve($a, $b);
}

fscanf(STDIN, "%d", $t);
for ($ks = 0; $ks < $t; $ks++) {
    fscanf(STDIN, "%d %d", $a, $b);
    fscanf(STDIN, "%d", $n);
    solve($a + 1, $b);
}
?>

```

Python2:

```

import sys

def solve(a, b):
    m = (a + b) / 2
    print m
    sys.stdout.flush()
    s = raw_input()
    if s == "CORRECT":
        return

```



```

elif s == "TOO_SMALL":
    a = m + 1
else:
    b = m - 1
solve(a, b)

T = input()
for _ in xrange(T):
    a, b = map(int, raw_input().split())
    _ = input()
    solve(a + 1, b)

```

Python3:

```

import sys

def solve(a, b):
    m = (a + b) // 2
    print(m)
    sys.stdout.flush()
    s = input()
    if s == "CORRECT":
        return
    elif s == "TOO_SMALL":
        a = m + 1
    else:
        b = m - 1
    solve(a, b)

T = int(input())
for _ in range(T):
    a, b = map(int, input().split())
    _ = int(input())
    solve(a + 1, b)

```

Ruby:

```

$stdout.sync = true

def solve(a, b)
    m = (a + b) / 2
    puts m
    $stdout.flush
    s = STDIN.gets.chomp
    if s.eql? "CORRECT"
        return
    elsif s.eql? "TOO_SMALL"
        solve(m + 1, b)
    else
        solve(a, m - 1)
    end
end

t = STDIN.gets.chomp.to_i

```

```
ks = 1
while ks <= t
  a, b = STDIN.gets.split.map &:to_i;
  n = STDIN.gets.chomp.to_i
  solve(a + 1, b)
  ks = ks + 1
end
```

Analysis: Senate Evacuation

Test set 1

With at most three parties and at most nine senators, various brute force approaches will work. One exhaustive strategy is to generate all possible different evacuation orders, treating senators from the same party as interchangeable, and then try all possible different ways of chopping those into groups of one or two senators.

Another, simpler strategy is to keep randomly choosing and trying one of the nine possible evacuations (A, B, C, AA, AB, AC, BB, BC, CC) as long as the chosen senator(s) exist and the evacuation will not cause a new absolute majority. You may worry that this strategy could get stuck, but the outcome of any legal evacuation will just be another possible test case for the problem, and the statement guarantees that every test case has a solution! With more parties and senators, though, this strategy might bog down in the details of checking the legality of evacuations, so we should come up with a more efficient approach.

Test set 2

Intuitively, it is safest to remove one senator at a time, and to always draw from whichever party has the most remaining senators (or any such largest party, if there is a tie). But this strategy won't always work! For example, if we have two senators from party A and two from party B, and no others, which is a valid test case, then removing one senator from either party will give the other party an absolute majority.

However, this strategy *is* always safe whenever there are more than two parties present. Suppose that party 1 is currently the largest, or tied for the largest, of at least three parties, and that we remove a single senator from party 1. Clearly, making party 1 smaller cannot give it an absolute majority that it didn't have before. But could some other party acquire an absolute majority as a result? Suppose that the removal of a senator from party 1 were to cause party 2, which currently has X senators, to have an absolute majority. But since party 1 was the largest, or tied for the largest, before a senator was removed, party 1 must still have at least $X-1$ senators. Moreover, since at least one more party is present, there is at least 1 other senator who is not from party 1 or 2. So there are a total of at least X remaining senators who are not from party 2, which means the X senators of party 2 are not enough to give it an absolute majority, so we have a contradiction.

If we start with three or more parties and keep evacuating a single senator from the largest party in this way, then at some point, we must reach a step in which we go from three parties to two parties. These two remaining parties must have only one senator each. Since we just removed the one remaining senator from the third party, it must have been a largest party, so the other two can be no larger. So we can remove this last pair of senators in a single evacuation as a final step.

What if we start with two parties? Since the problem statement guarantees that no party begins with a majority, these parties must have equal numbers of senators. So, we can evacuate them in pairs, one from each party, until the evacuation is complete.

This approach takes more steps than are needed — most of those single evacuations can be paired up — but it gets the job done.

Analysis: Steed 2: Cruise Control

Pop quiz, hotshots! This problem seems pretty complicated at first glance. What do you do?

One natural strategy is to try binary searching on Annie's speed, but it is difficult to directly determine whether a given speed avoids passing another horse; the input data alone does not tell us where each horse is at any given time, because horses might slow other horses down. In theory, we could figure out when faster horses catch up to slower horses and slow down, determine the exact path of each horse, and check whether our chosen speed crosses any of those paths. With only up to two horses in test set 1, this sort of calculation is feasible, but it would be laborious for test set 2.

However, we can avoid all of that work via some observations. To maximize cruising speed, Annie's horse should reach the destination at exactly the same time as the horse ahead of her (let's call it Horse A); there is no reason to leave a gap. Either Horse A will reach the destination without having to slow down (and so it will be the one that directly limits Annie's speed), or it will be slowed down at some point by the horse ahead of it (let's call it Horse B). The same is true for Horse B: either it will never have to slow down (and so it will be the one that ultimately limits Annie's speed), or it will be slowed down by the horse ahead of *it*, and so on. So there will be a single "limiting horse" on the road that ultimately determines how fast Annie's horse can reach the destination. We claim that this "limiting horse" is the only horse that matters, and we can disregard all of the others!

It is easy to see that we can ignore the horses to the east of the limiting horse; they will reach and pass the destination before the limiting horse gets there. What about the "intermediate horses" between Annie and the limiting horse? We know from the way we have defined the limiting horse that every intermediate horse will catch up to the limiting horse before reaching the destination. (If one did not, then *it* would be the limiting horse.) Suppose that Annie chooses a cruising speed that gets her to the destination at exactly the same time as the limiting horse. We certainly cannot go faster than this. Moreover, this speed is safe: it cannot possibly cause Annie to pass any of the intermediate horses. If she were going fast enough to overtake an intermediate horse, then she would definitely be going fast enough to pass the limiting horse, since every intermediate horse will catch up to the limiting horse. This would cause a contradiction. Therefore, we do not need to worry about the intermediate horses or their interactions with each other.

So, once we have identified the limiting horse, the strategy is simple: go at the exact speed that will cause Annie to reach the destination at the same time as the limiting horse. This speed can be found in constant time. We could identify the limiting horse directly via the argument in our third paragraph above, but even this would be unnecessary work. Instead, for each horse in turn, we can pretend that it is the limiting horse and calculate the cruising speed that it would force. Then the smallest of those speeds is our answer. (If any horse allows a faster cruising speed than another, it cannot be the limiting horse, because that cruising speed would cause Annie to pass the true limiting horse.) This takes $O(N)$ time.